

Assignment 1 : Python Basics

Q1. Explain the key features of Python that make it a popular choice for programming.

Ans:- Python is one of the most popular languages globally because of its simplicity and versatility. Its key features include:

1. Easy to Read and Learn: Its syntax is very close to English, making it beginner-friendly.
2. Interpreted Language: Code is executed line-by-line, which makes debugging much faster.
3. Dynamically Typed: You don't need to declare the type of a variable (like int or string) when you create it; Python figures it out automatically.
4. Extensive Libraries: It has a massive ecosystem of pre-written code for Data Science (NumPy, Pandas), AI, and Web Development.

```
In [1]: '''Code Example : '''  
  
# Dynamic typing: no need to define 'int' or 'str'  
x = 10  
y = "Hello World"  
  
print(type(x)) # Output: <class 'int'>  
print(type(y)) # Output: <class 'str'>
```

```
<class 'int'>  
<class 'str'>
```

Q2. Describe the role of predefined keywords in Python and provide examples of how they are used in a program.

Ans:- Keywords are reserved words in Python that have special meaning to the interpreter. They form the basic vocabulary and grammar of the language. Because they are reserved for specific tasks (like logic, loops, or defining functions), they cannot be used as names for variables, functions, or any other identifiers.

- Role: They tell Python how to structure the logic of the code (e.g., if defines a condition, def defines a function).

- Case Sensitivity: Most keywords are lowercase (like if, while, import), but some start with a capital letter (True, False, None).

Common Examples:

- Control Flow: if, elif, else, for, while, break.
- Boolean/Logic: True, False, and, or, not.
- Structure: def, return, class, import, pass.

```
In [2]: '''Code Example : '''

# 'import' is a keyword used to bring in Libraries
import math

# 'def', 'if', 'return', and 'False' are all keywords here
def check_even(number):
    if number % 2 == 0:
        return True
    else:
        return False

result = check_even(10)
print(result) # Output: True
```

True

Q3. Compare and contrast mutable and immutable objects in Python with example.

Ans:- In Python, every variable is an object. These objects are classified into two categories based on whether their content can be changed after they are created:

- Mutable Objects: These objects can be modified after they are created without changing their identity (memory address). You can add, remove, or change items inside them.

Examples: list, dict, set, bytearray.

- Immutable Objects: These objects cannot be changed once they are assigned. If you try to modify an immutable object, Python actually creates a brand-new object in memory with the new value.

Examples: int, float, string, tuple, bool.

```
In [3]: '''Code Example'''

# --- MUTABLE EXAMPLE (List) ---
my_list = [1, 2, 3]
print("Original List ID:", id(my_list))

my_list.append(4) # Modifying the list
print("Modified List:", my_list)
print("Same ID after modification:", id(my_list)) # The ID remains the same

# --- IMMUTABLE EXAMPLE (String) ---
my_string = "Hello"
print("\nOriginal String ID:", id(my_string))

# Attempting to change 'Hello' to 'Hello!'
my_string = my_string + "!"
print("New String:", my_string)
print("New ID (Different from original):", id(my_string)) # Python created a new object
```

Original List ID: 2642043565888

Modified List: [1, 2, 3, 4]

Same ID after modification: 2642043565888

Original String ID: 2642043578976

New String: Hello!

New ID (Different from original): 2642043572640

Q4. Discuss the different types of operators in Python and provide examples of how they are used.

Ans:-Python operators are categorized based on the type of operation they perform. Here are the five most essential categories:

1. Arithmetic Operators: Used for mathematical calculations.

+, -, *, / (division), % (modulus/remainder), (exponentiation), // (floor division).

2. Comparison (Relational) Operators: Used to compare two values; they always return True or False.

==, !=, >, <, >=, <=.

3. Logical Operators: Used to combine conditional statements.

and, or, not.

4. Assignment Operators: Used to assign values to variables.

=, +=, -=, *=, /=.

5. Membership Operators: Used to test if a sequence is present in an object.

in, not in.

In [4]: `'''Code Example '''`

```
# 1. Arithmetic
print(10 % 3)    # Output: 1 (Remainder of 10/3)
print(2 ** 3)    # Output: 8 (2 raised to the power 3)

# 2. Comparison
print(10 > 5)    # Output: True

# 3. Logical
a = True
b = False
print(a and b)   # Output: False (Both must be True)

# 4. Assignment
x = 5
x += 2           # Same as x = x + 2
print(x)         # Output: 7

# 5. Membership
```

```
fruits = ["apple", "banana"]  
print("cherry" in fruits) # Output: False
```

```
1  
8  
True  
False  
7  
False
```

Q5. Explain the concept of type casting in Python with examples.

Ans : Type casting is the method used to change the data type of a variable from one form to another (e.g., converting a string to an integer).

There are two main types in Python:

1. Implicit Type Conversion: Python automatically converts one data type to another to prevent data loss. This usually happens when you perform an operation involving an integer and a float; Python promotes the integer to a float.
2. Explicit Type Conversion (Type Casting): The programmer manually converts the data type using built-in functions. This is necessary when Python cannot automatically determine how to handle the data (e.g., adding a string "10" to an integer 5).

Common Functions:

- int(): Converts to an integer.
- float(): Converts to a decimal number.
- str(): Converts to a string.
- list(): Converts a sequence to a list.

```
In [5]: '''Code Example : '''  
  
# 1. Implicit Conversion  
num_int = 10  
num_float = 1.5  
sum_val = num_int + num_float
```

```

print(f"Value: {sum_val}, Type: {type(sum_val)}")
# Output: Value: 11.5, Type: <class 'float'>

# 2. Explicit Conversion (Type Casting)
price_str = "199"
# Convert string to int to perform math
total_price = int(price_str) + 50

print(f"Total: {total_price}")
# Output: Total: 249

# 3. Converting to String for concatenation
age = 21
message = "I am " + str(age) + " years old."
print(message)

```

Value: 11.5, Type: <class 'float'>

Total: 249

I am 21 years old.

Q6. How do conditional statements work in Python? Illustrate with example

Ans: Conditional statements allow a program to make decisions and execute specific blocks of code based on whether a condition is True or False. In Python, this is handled by the if, elif (else if), and else keywords.

Python uses indentation (usually 4 spaces) to define which lines of code belong to which branch of the condition.

- if: The starting point. If the condition is True, the code inside runs.
- elif: Short for "else if." It allows you to check multiple alternative conditions if the previous ones were False.
- else: The "catch-all." It runs only if none of the above conditions were met.

In [6]: `'''Code Example : '''`

```

# A program to check the grade based on marks
marks = 85

```

```
if marks >= 90:
    print("Grade: A+")
elif marks >= 80:
    print("Grade: A")
elif marks >= 70:
    print("Grade: B")
else:
    print("Grade: C or below")

# Since marks is 85, this will output: Grade: A
```

Grade: A

Q7. Describe the different types of loops in Python and their use cases with examples.

Ans : Loops are fundamental control structures that allow you to execute a block of code repeatedly. Python provides two primary types of loops, each suited for different scenarios:

1. The for Loop The for loop is used for definite iteration. It iterates over a sequence (such as a list, tuple, string, or range) and executes the code block for each element in that sequence.
 - Use Case: Use a for loop when you know in advance how many times the code needs to run (e.g., "Do this for every item in this list" or "Do this exactly 10 times").
2. The while Loop The while loop is used for indefinite iteration. It continues to execute as long as a specified condition remains True.
 - Use Case: Use a while loop when the number of iterations is not known beforehand and depends on a dynamic condition (e.g., "Keep asking for a password until it is correct" or "Keep running until the battery level is low").

```
In [7]: ''' Code Example : '''

# --- FOR LOOP EXAMPLE ---
# Use case: Iterating through a fixed list of names
students = ["Aniket", "Rahul", "Priya"]

print("For Loop - Student List:")
for name in students:
    print(f"Processing record for: {name}")
```

```
# --- WHILE LOOP EXAMPLE ---  
# Use case: Decrementing a value until a threshold is reached  
countdown = 5  
  
print("\nWhile Loop - Countdown:")  
while countdown > 0:  
    print(f"Starting in... {countdown}")  
    countdown -= 1 # Crucial: update the condition variable  
  
print("Action!")
```

For Loop - Student List:

Processing record for: Aniket

Processing record for: Rahul

Processing record for: Priya

While Loop - Countdown:

Starting in... 5

Starting in... 4

Starting in... 3

Starting in... 2

Starting in... 1

Action!